

Coding Documentation

Human Development Index (HDI) Prediction Using Machine Learning

Student Name: Egala Durga Rao
Department: Computer Science and Engineering (Data Science)
Date: 05 July 2026

1. Introduction

Coding documentation provides a detailed explanation of the source code used to develop the Human Development Index (HDI) Prediction System. It helps developers, students, and evaluators understand the structure of the project, the purpose of each module, and the workflow of the application.

Well-documented code improves readability, maintainability, debugging, and future enhancements.

2. Programming Language

The project was developed using the following technologies.

Technology	Purpose
Python	Backend Programming
Flask	Web Framework
HTML	User Interface
CSS	Styling
Joblib	Load Machine Learning Model
Pandas	Data Processing
NumPy	Numerical Operations
Scikit-learn	Machine Learning
Matplotlib	Visualization
Seaborn	Statistical Graphs

3. Project Folder Structure

```
HDI-Prediction-Using-ML
├── App
│   ├── app.py
│   ├── templates
│   │   └── index.html
│   └── static
│       └── style.css
├── Dataset
│   └── HDI.csv
├── Images
├── Model
│   └── HDI_LinearRegression_Model.pkl
├── Notebook
│   └── HDI_Model.ipynb
├── Documentation
├── README.md
├── requirements.txt
└── LICENSE
```

4. Description of Files

app.py

This is the main backend file of the Flask application.

Responsibilities:

- Starts Flask server
- Loads trained model
- Receives user input
- Performs prediction
- Returns prediction result

index.html

Provides the user interface.

Responsibilities:

- Accept user inputs

- Display prediction
 - Show error messages
 - Responsive design
-

style.css

Responsible for visual appearance.

Features include:

- Responsive layout
 - Modern design
 - Gradient colors
 - Buttons
 - Cards
 - Animations
-

HDI_Model.ipynb

Contains complete Machine Learning implementation.

Includes:

- Dataset loading
 - Data preprocessing
 - Data visualization
 - Model training
 - Model comparison
 - Evaluation
 - Saving trained model
-

HDI.csv

Contains the training dataset.

Columns include:

- Country
 - Life Expectancy
 - Mean Years of Schooling
 - Expected Years of Schooling
 - GNI per Capita
 - HDI
-

HDI_LinearRegression_Model.pkl

Stores the trained Linear Regression model using Joblib.

5. Application Workflow

The overall coding workflow is shown below.

```
graph TD;
    User --> HTML_Form[HTML Form];
    HTML_Form --> Flask_Route[Flask Route];
    Flask_Route --> Input_Validation[Input Validation];
    Input_Validation --> Load_Model[Load Model];
    Load_Model --> Predict_HDI[Predict HDI];
    Predict_HDI --> Classify_Category[Classify Category];
    Classify_Category --> Display_Result[Display Result];
```

6. Important Libraries

Flask

Used to develop the web application.

Main functions:

- Flask()
- render_template()
- request()

Pandas

Used for:

- Reading CSV files
- Data preprocessing
- Data manipulation

NumPy

Used for:

- Numerical arrays
 - Mathematical calculations
-

Scikit-learn

Used for:

- Linear Regression
 - Decision Tree
 - Random Forest
 - Model Evaluation
 - Train-Test Split
-

Joblib

Used to:

- Save trained model
 - Load model quickly
-

7. Machine Learning Workflow

```
Load Dataset
  ↓
Data Cleaning
  ↓
EDA
  ↓
Feature Selection
  ↓
Train-Test Split
  ↓
Model Training
  ↓
Model Evaluation
  ↓
Save Model
  ↓
Flask Integration
```

8. Prediction Process

When the user submits the form:

Step 1

Read user inputs.

Step 2

Validate values.

Step 3

Load trained model.

Step 4

Convert inputs into numerical format.

Step 5

Predict HDI.

Step 6

Identify development category.

Step 7

Display result.

9. Error Handling

The application performs several validations.

- Empty input fields
- Invalid numeric values
- Negative numbers
- Incorrect data types

Appropriate messages are displayed to guide the user.

10. Coding Standards Followed

The project follows standard programming practices.

- Proper folder organization
- Modular code
- Meaningful variable names
- Reusable functions
- Simple logic

- Comments where necessary
 - Responsive frontend
 - Separate backend and frontend
-

11. Advantages of the Code Structure

- Easy to understand
 - Beginner-friendly
 - Easy debugging
 - Modular architecture
 - Reusable components
 - Easy future enhancement
 - Suitable for deployment
-

12. Future Improvements

The coding structure can be enhanced by:

- REST API integration
 - Database connectivity
 - User authentication
 - Docker support
 - Cloud deployment
 - Multiple ML models
 - Streamlit interface
-

13. Conclusion

The coding documentation explains the complete implementation of the Human Development Index Prediction System. The project follows a modular architecture where each component has a specific responsibility. Flask handles the web interface, the trained Linear Regression model performs predictions, and HTML/CSS provide a responsive user experience.

This structured coding approach improves readability, simplifies maintenance, and supports future enhancements, making the project suitable for both academic purposes and real-world deployment.